

Get Ready to Code

A 15-minute check to do **before the first session**. You already built pages in the HTML & CSS course, so most of this is confirming your tools still work — plus one new skill: opening the browser's **DevTools console**, where JavaScript talks back to you.

 ~15 min

No JavaScript experience required

Any computer: Windows / Mac / Linux

PLEASE FINISH THIS BEFORE CLASS

Class time is for coding, not installing. If you get stuck on any step, that's fine — note where, and we'll help you in the first few minutes. But please **try** before you arrive.

WHAT TO BRING

- ◆ A **laptop** (Windows, macOS, or Linux) and its **charger**.
- ◆ **VS Code with the Live Server extension** — the same setup you used in the HTML & CSS course.
- ◆ An up-to-date **modern browser**. **Chrome or Edge** is recommended (class demos use them); Firefox works fine too.
- ◆ What you know about HTML and CSS. That's it — everything else is provided.

SET UP IN 5 STEPS

1 Confirm VS Code & Live Server

~3 min

- 1 Open **VS Code**. If it isn't installed, download it free from **code.visualstudio.com** and accept the installer's defaults.
- 2 Click the **Extensions** icon in the left sidebar (four squares) and search for **Live Server** (by Ritwick Dey).
- 3 If it shows **Install**, click it. If it already shows **Uninstall** or a gear icon, you're set.

IF LIVE SERVER WON'T INSTALL OR START

Install Microsoft's **Live Preview** extension instead. It does the same job: right-click the file → **Show Preview** (or open it in your browser from the preview's menu).

2 Update your browser

~2 min

- 1 **Chrome:** menu (⋮) → **Help** → **About Google Chrome**. **Edge:** menu (⋯) → **Help and feedback** → **About Microsoft Edge**. **Firefox:** menu (≡) → **Help** → **About Firefox**.
- 2 Let it download any update, then click **Relaunch**.

WHY BOTHER?

We use modern JavaScript features such as `fetch`, `async` / `await`, and optional chaining. Every current browser supports them; a browser that hasn't updated in years may not.

3 Learn to open the console

~5 min

The **console** lives in the browser's **DevTools**. You'll open it dozens of times a day, so learn the shortcut for your system now.

BROWSER	WINDOWS / LINUX	MACOS
Chrome	Ctrl + Shift + J	Cmd + Option + J
Edge	Ctrl + Shift + J	Cmd + Option + J
Firefox	Ctrl + Shift + K	Cmd + Option + K
Any browser	F12, then click the Console tab	Right-click the page → Inspect , then Console

- 1 Open any web page and press the shortcut. A panel opens with a **Console** tab and a blinking cursor next to `>`.
- 2 Type `2 + 2` and press **Enter**. The console answers `4`. That's JavaScript running.

ON A MAC WITH SAFARI?

Safari hides its DevTools until you turn them on: **Safari → Settings → Advanced → tick Show features for web developers** (older versions: **Show Develop menu in menu bar**). Then press **Cmd + Option + C**. It works, but installing Chrome or Edge will make class easier to follow.

4 Create a practice folder

~2 min

- 1 On your Desktop (or in your Documents folder), create a new folder named `js-practice` (lowercase, no spaces).
- 2 In VS Code, choose **File → Open Folder...** and select it.
- 3 This is where your exercises and labs will live. Keep one sub-folder per exercise as the week goes on.

ONE RULE THAT SAVES HEADACHES

Keep file and folder names **lowercase with no spaces**, and match them **exactly** in your code.

`src="app.js"` will not load a file named `App.js` on many servers.

5 The 2-minute "hello console" test

~2 min

- 1 In `js-practice`, create `index.html`. Type `!` and press **Tab** for the HTML skeleton, then put this inside `<body>`:

```
<h1>Hello!</h1>
<script src="app.js"></script>
```

- 2 Create a second file in the same folder, `app.js`, containing:

```
console.log("Hello, console!");
document.querySelector("h1").textContent = "JavaScript is working!";
```

- 3 Save both files (Ctrl/⌘ + S), right-click `index.html` → **Open with Live Server**.

- 4 The page heading should read "JavaScript is working!". Open the console (step 3): you should see Hello, console!

✓ YOU'RE READY

If you saw both — the changed heading **and** the console message — you're completely set up for Day 1. If the heading still says "Hello!", check that `app.js` is in the same folder and spelled exactly the same, and look for a red error in the console. Still stuck? Note what you saw and we'll sort it out in the first few minutes of class.

FREQUENTLY ASKED

◆ DO I NEED TO KNOW ANY PROGRAMMING?

No. We start JavaScript from zero. You only need the HTML and CSS you already know.

◆ DO I NEED TO BUY OR SIGN UP FOR ANYTHING?

No. Every tool is free, and the public data services we use later in the week don't need an account or a key.

◆ CAN I USE A CHROMEBOOK OR TABLET?

A full laptop (Windows/Mac/Linux) is strongly recommended. Chromebooks can work but VS Code setup is trickier — contact us first if that's all you have.

◆ WHAT IF I CAN'T FINISH THE SETUP?

Come anyway. Note where you got stuck; we'll help you in the first few minutes so you're coding with everyone else.

FOR THE HOST OR ORGANIZER: NETWORK CHECK

The room needs ordinary internet access. On Day 5 students call public APIs from their own browsers:

`api.github.com` (Lesson 20) and `dummyjson.com` (Lesson 21); the final assessment may also use `openlibrary.org` and `open-meteo.com`. Make sure a content filter doesn't block them. **GitHub allows about 60 requests per hour per public IP address without logging in** — and a classroom behind one network usually shares a single public IP, so the whole class shares those 60. Ask IT whether the room has one shared IP; if it does, have students test sparingly, keep a phone hotspot as a backup, and expect a friendly "rate limit reached" message rather than a bug if the budget runs out.